Exercise-2

Aim: Write a Program to implement the data link layer farming methods such as

i) Character stuffing ii) bit stuffing

PROGRAM:

```
#include <stdio.h>
int main() {
    int a[20]; // Increased size to handle extra bits after stuffing
    int i, j, n, c = 0, pos = 0;

    printf("\nEnter the number of bits: ");
    scanf("%d", &n);

    if (n > 15) {
        printf("Error: Maximum 15 bits allowed.\n");
        return 1;
    }

    printf("\nEnter the bits (0 or 1):\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        if (a[i] != 0 && a[i] != 1) {
            printf("Error: Only 0 and 1 are allowed.\n");
```

```
            return 1;
        }
}

for (i = 0; i < n; i++) {
        if (a[i] == 1) {
            c++;
            if (c == 5) {
                pos = i + 1;
                for (j = n; j > pos; j--) {
                    a[j] = a[j - 1];
                }
                a[pos] = 0; // Insert 0 after five consecutive 1s
                n++;
                c = 0; // Reset the counter
            }
        } else {
            c = 0; // Reset the counter for consecutive 1s
        }
}

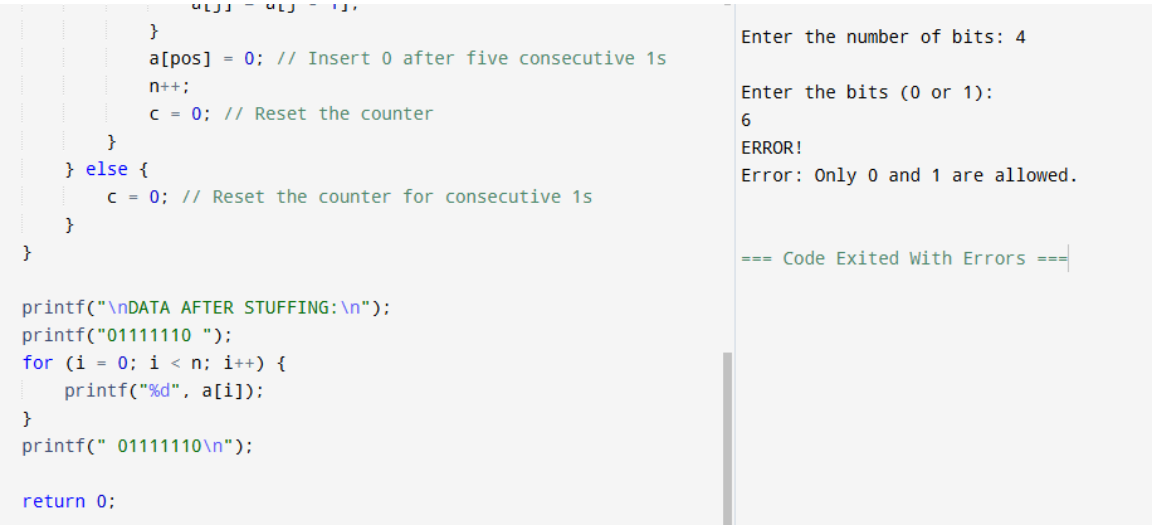
printf("\nDATA AFTER STUFFING:\n");
printf("01111110 ");
```

```
    for (i = 0; i < n; i++) {
        printf("%d", a[i]);
    }
    printf(" 01111110\n");

    return 0;
}
```

output:

case 1:



```
            }
            a[pos] = 0; // Insert 0 after five consecutive 1s
            n++;
            c = 0; // Reset the counter
        }
    } else {
        c = 0; // Reset the counter for consecutive 1s
    }
}

printf("\nDATA AFTER STUFFING:\n");
printf("01111110 ");
for (i = 0; i < n; i++) {
    printf("%d", a[i]);
}
printf(" 01111110\n");

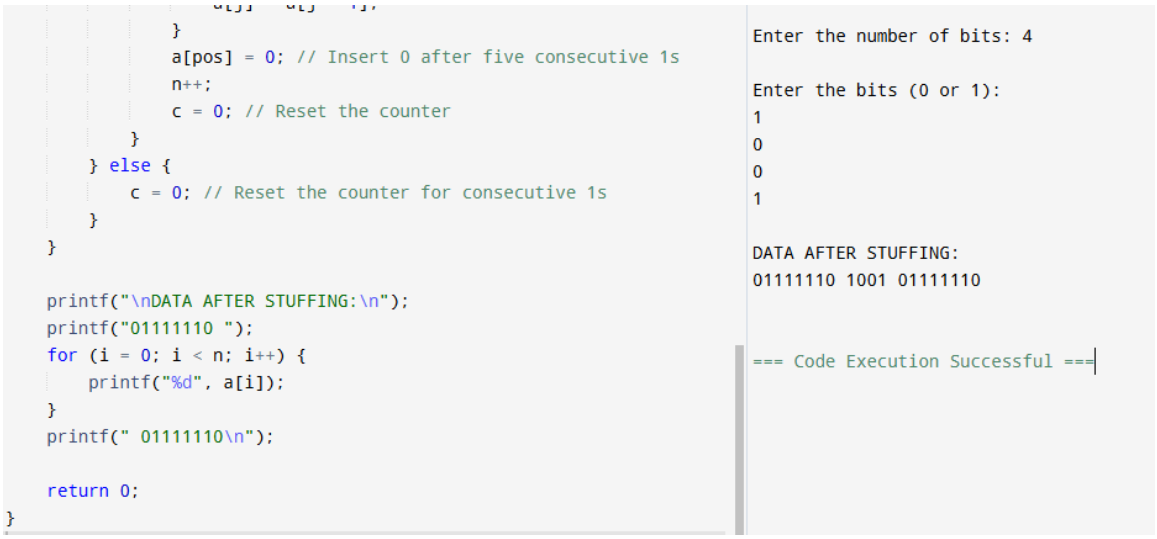
return 0;
```

```
Enter the number of bits: 4

Enter the bits (0 or 1):
6
ERROR!
Error: Only 0 and 1 are allowed.


=== Code Exited With Errors ===
```

case 2:



```
            }
            a[pos] = 0; // Insert 0 after five consecutive 1s
            n++;
            c = 0; // Reset the counter
        }
    } else {
        c = 0; // Reset the counter for consecutive 1s
    }
}

printf("\nDATA AFTER STUFFING:\n");
printf("01111110 ");
for (i = 0; i < n; i++) {
    printf("%d", a[i]);
}
printf(" 01111110\n");

return 0;
}
```

```
Enter the number of bits: 4

Enter the bits (0 or 1):
1
0
0
1

DATA AFTER STUFFING:
01111110 1001 01111110


=== Code Execution Successful ===
```

Aim:

Implement the data link layer framing methods such as Character Stuffing and also De-stuff it

program:

// Online C compiler to run C program online

#include <stdio.h>

#include<stdlib.h>

void charc(void);

void main()

{

int choice;

while(1)

{

printf("\n\n\n1.character stuffing");

printf("\n\n2.exit");

printf("\n\n\nenter choice:");

scanf("%d",&choice);

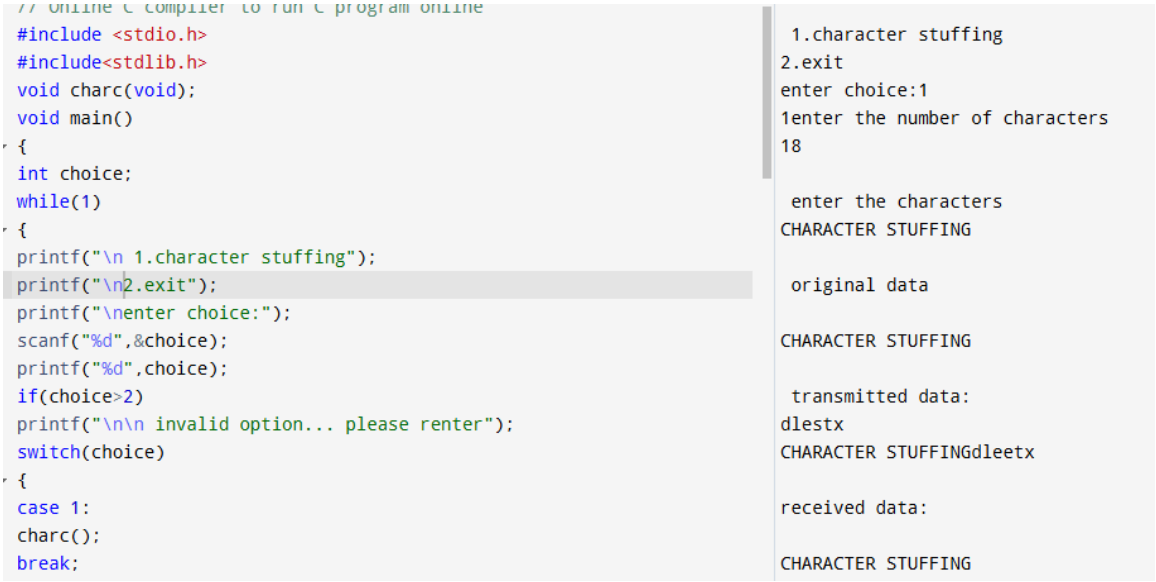
```
printf("%d",choice);
if(choice>2)
printf("\n\n invalid option... please renter");
switch(choice)
{
case 1:
charc();
break;
case 2:
exit(0);
}
}
}
void charc(void)
{
char c[50],d[50],t[50];
int i,m,j;
//clrscr();
printf("enter the number of characters\n");
scanf("%d",&m);
printf("\n enter the characters\n");
for(i=0;i<m+1;i++)
{
scanf("%c",&c[i]);
}
```

```
printf("\n original data\n");
for(i=0;i<m+1;i++)
printf("%c",c[i]);
d[0]='d';
d[1]='l';
d[2]='e';
d[3]='s';
d[4]='t';
d[5]='x';
for(i=0,j=6;i<m+1;i++,j++)
{
if((c[i]=='d'&&c[i+1]=='l'&& c[i+2]=='e'))
{
d[j]='d';
j++;
d[j]='l';
j++;
d[j]='e';
j++;
m=m+3;
}
d[j]=c[i];
}
m=m+6;
m++;
```

```
d[m]='d';
m++;
d[m]='l';
m++;
d[m]='e';
m++;
d[m]='e';
m++;
d[m]='t';
m++;
d[m]='x';
m++;
printf("\n\n transmitted data: \n");
for(i=0;i<m;i++)
{
printf("%c",d[i]);
}
for(i=6,j=0;i<m-6;i++,j++)
{
if(d[i]=='d'&&d[i+1]=='l'&&d[i+2]=='e'&&d[i+3]=='d'&&d[i+4]=='l'&&d[i+5]=='e')
i=i+3;
t[j]=d[i];
}
printf("\n\nreceived data:\n");
```

```
for(i=0;i<j;i++)
{
printf("%c",t[i]);
}
}
```

Output:



```
// Online C compiler to run C program online
#include <stdio.h>
#include<stdlib.h>
void charc(void);
void main()
{
int choice;
while(1)
{
printf("\n 1.character stuffing");
printf("\n2.exit");
printf("\nenter choice:");
scanf("%d",&choice);
printf("%d",choice);
if(choice>2)
printf("\n\n invalid option... please renter");
switch(choice)
{
case 1:
charc();
break;
```

```
 1.character stuffing
2.exit
enter choice:1
1enter the number of characters
18

 enter the characters
CHARACTER STUFFING

 original data

CHARACTER STUFFING

 transmitted data:
dlestx
CHARACTER STUFFINGdleetx

received data:

CHARACTER STUFFING
```

Exercise-3

Aim:Write a Program to implement data link layer farming method checksum

Program:

```
// checksum ex-3
#include<stdio.h>
//#include<conio.h>
#include<math.h>
int sender(int b[10],int k) {
int checksum,sum=0,i;
```

```
printf("\n****SENDER****\n");
for(i=0;i<k;i++)
sum+=b[i];
printf("SUM IS: %d",sum);
checksum=~sum;
printf("\nSENDER's CHECKSUM IS:%d",checksum);
return checksum;
}
int receiver(int c[10],int k,int scheck)
{
int checksum,sum=0,i;
printf("\n\n****RECEIVER****\n");
for(i=0;i<k;i++)
sum+=c[i];
printf(" RECEIVER SUM IS:%d",sum);
sum=sum+scheck;
checksum=~sum;
printf("\nRECEIVER's CHECKSUM IS:%d",checksum);
return checksum;
}
void main()
{
int a[10],i,m,scheck,rcheck;
//clrscr();outdated in turbo c
printf("\nENTER SIZE OF THE STRING:");
```

```
scanf("%d",&m);
printf("\nENTER THE ELEMENTS OF THE ARRAY:");
for(i=0;i<m;i++)
scanf("%d",&a[i]);
scheck=sender(a,m);
rcheck=receiver(a,m,scheck);
if(rcheck==0)
printf("\n\nNO ERROR IN TRANSMISSION\n\n");
else
printf("\n\nERROR DETECTED");
//getch();outdated in turbo c
}
```

```
// checksum ex-3
#include<stdio.h>
//#include<conio.h>
#include<math.h>
int sender(int b[10],int k) {
int checksum,sum=0,i;
printf("\n****SENDER****\n");
for(i=0;i<k;i++)
sum+=b[i];
printf("SUM IS: %d",sum);
checksum=~sum;
printf("\nSENDER's CHECKSUM IS:%d",checksum);
return checksum;
}
int receiver(int c[10],int k,int scheck)
{
int checksum,sum=0,i;
printf("\n\n****RECEIVER****\n");
for(i=0;i<k;i++)
```

```
ENTER SIZE OF THE STRING:5

ENTER THE ELEMENTS OF THE ARRAY:1
2
3
4
5

****SENDER****
SUM IS: 15
SENDER's CHECKSUM IS:-16

****RECEIVER****
 RECEIVER SUM IS:15
RECEIVER's CHECKSUM IS:0

NO ERROR IN TRANSMISSION
```

Exercise-4

Aim:Write a program for Hamming Code generation for error detection and correction

```
#include <stdio.h>
#include <math.h>
int input[32];
```

```
int code[32];
// Function to calculate parity for a given position
int ham_calc(int position, int c_l) {
    int count = 0, i, j;
    i = position - 1;
    while (i < c_l) {
        for (j = i; j < i + position && j < c_l; j++) {
            if (code[j] == 1) {
                count++;
            }
        }
        i += 2 * position;
    }
    return count % 2;
}

void main() {
    int n, i, p_n = 0, c_l, j, k, error_pos;

    printf("Please enter the length of the Data Word: ");
    scanf("%d", &n);

    printf("Please enter the Data Word (0s and 1s only):\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &input[i]);
```

```
}
// Calculate the number of parity bits required
i = 0;
while (n > (int)pow(2, i) - (i + 1)) {
    p_n++;
    i++;
}
c_l = p_n + n; // Code length = parity bits + data bits
j = k = 0;
// Initialize the code array with data and parity placeholders
for (i = 0; i < c_l; i++) {
    if (i == ((int)pow(2, k) - 1)) {
        code[i] = 0; // Parity bit placeholder
        k++;
    } else {
        code[i] = input[j];
        j++;
    }
}

// Calculate parity bits
for (i = 0; i < p_n; i++) {
    int position = (int)pow(2, i);
    int value = ham_calc(position, c_l);
    code[position - 1] = value;
```

```
}

printf("\nThe calculated Code Word is: ");
for (i = 0; i < c_l; i++) {
    printf("%d", code[i]);
}
printf("\n");

// Accept the received code word
printf("Please enter the received Code Word:\n");
for (i = 0; i < c_l; i++) {
    scanf("%d", &code[i]);
}

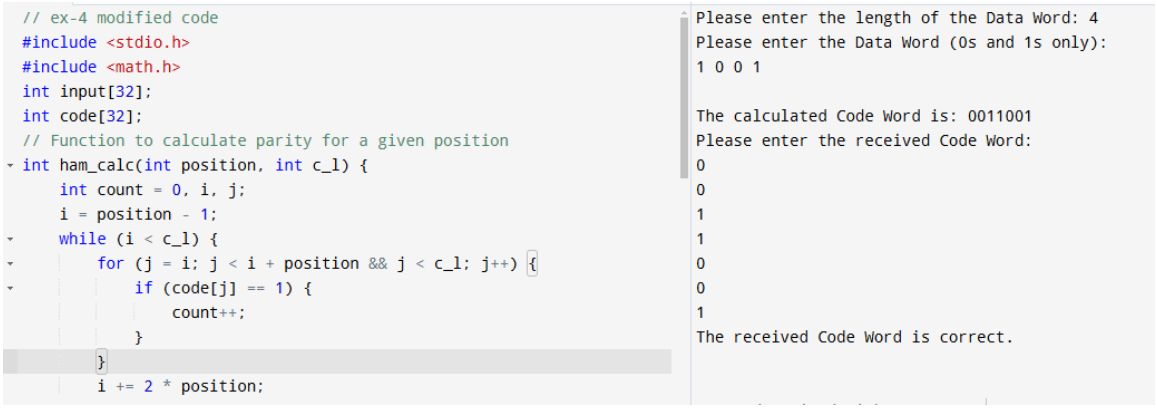
// Check for errors in the received code word
error_pos = 0;
for (i = 0; i < p_n; i++) {
    int position = (int)pow(2, i);
    int value = ham_calc(position, c_l);
    if (value != 0) {
        error_pos += position;
    }
}

if (error_pos == 0) {
```

```
        printf("The received Code Word is correct.\n");
    } else {
        printf("Error at bit position: %d\n", error_pos);
    }
}
```

Output:

Case 1:

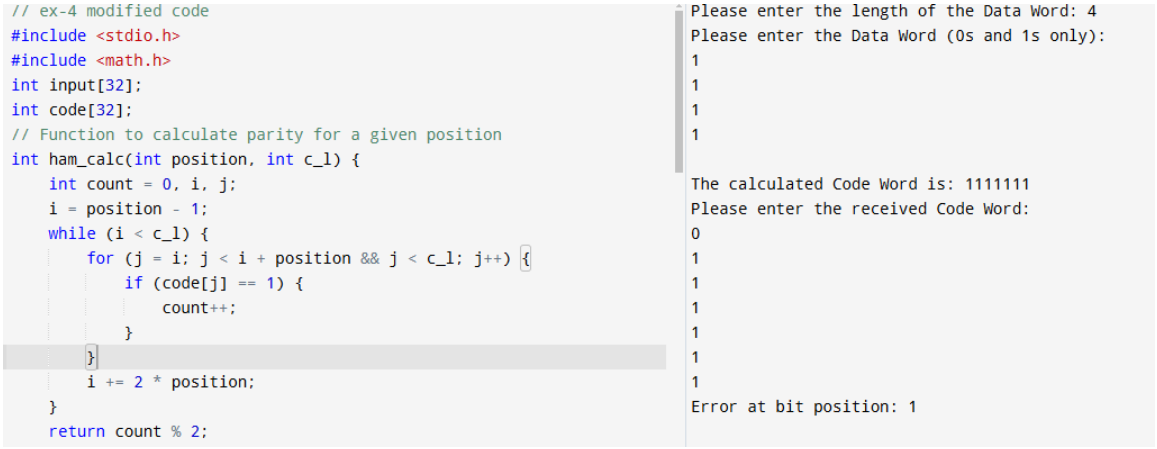


```
// ex-4 modified code
#include <stdio.h>
#include <math.h>
int input[32];
int code[32];
// Function to calculate parity for a given position
int ham_calc(int position, int c_l) {
    int count = 0, i, j;
    i = position - 1;
    while (i < c_l) {
        for (j = i; j < i + position && j < c_l; j++) {
            if (code[j] == 1) {
                count++;
            }
        }
        i += 2 * position;
```

```
Please enter the length of the Data Word: 4
Please enter the Data Word (0s and 1s only):
1 0 0 1

The calculated Code Word is: 0011001
Please enter the received Code Word:
0
0
1
1
0
0
1
The received Code Word is correct.
```

Case 2:



```
// ex-4 modified code
#include <stdio.h>
#include <math.h>
int input[32];
int code[32];
// Function to calculate parity for a given position
int ham_calc(int position, int c_l) {
    int count = 0, i, j;
    i = position - 1;
    while (i < c_l) {
        for (j = i; j < i + position && j < c_l; j++) {
            if (code[j] == 1) {
                count++;
            }
        }
        i += 2 * position;
    }
    return count % 2;
```

```
Please enter the length of the Data Word: 4
Please enter the Data Word (0s and 1s only):
1
1
1
1

The calculated Code Word is: 1111111
Please enter the received Code Word:
0
1
1
1
1
1
1
Error at bit position: 1
```

Exercise-5

Aim: Write a Program to implement on a data set of characters the three CRC polynomials – CRC 12, CRC 16

and CRC CCIP.

Program:

```
// crc ex-5
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void perform_crc(int frame[], int gen[], int rem[], int frame_len, int gen_len) {
    memcpy(rem, frame, frame_len * sizeof(int));
    for (int i = 0; i <= frame_len - gen_len; i++) {
        if (rem[i] == 1) {
            for (int j = 0; j < gen_len; j++) {
                rem[i + j] ^= gen[j];
            }
        }
    }
}

int main() {
    int frame[50] = {0}, gen[10] = {0}, rem[50] = {0}, frame_len, gen_len;

    printf("Enter the degree of the data polynomial: ");
    scanf("%d", &frame_len);
    frame_len++;

    printf("Enter the data polynomial coefficients (binary): ");
```

```
for (int i = 0; i < frame_len; i++) scanf("%d", &frame[i]);

printf("Enter the degree of the generator polynomial: ");
scanf("%d", &gen_len);
gen_len++;

printf("Enter the generator polynomial coefficients (binary): ");
for (int i = 0; i < gen_len; i++) scanf("%d", &gen[i]);

// Append zeros for CRC calculation
int total_len = frame_len + gen_len - 1;
for (int i = frame_len; i < total_len; i++) frame[i] = 0;

printf("\nFrame after appending zeros: ");
for (int i = 0; i < total_len; i++) printf("%d", frame[i]);

perform_crc(frame, gen, rem, total_len, gen_len);

printf("\nRemainder: ");
for (int i = frame_len; i < total_len; i++) {
    printf("%d", rem[i]);
    frame[i] = rem[i]; // Append remainder to frame
}

printf("\nTransmitted Frame: ");
```

```
    for (int i = 0; i < total_len; i++) printf("%d", frame[i]);

    // Simulate receiving the frame and check for errors
    printf("\n\nEnter the received frame: ");
    for (int i = 0; i < total_len; i++) scanf("%d", &frame[i]);

    perform_crc(frame, gen, rem, total_len, gen_len);

    int error = 0;
    for (int i = frame_len; i < total_len; i++) {
        if (rem[i] != 0) error = 1;
    }

    if (error)
        printf("\nError detected in the received frame.\n");
    else
        printf("\nNo error detected. Data sent successfully.\n");

    return 0;
}
```

Output:

```c
// crc ex-5
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void perform_crc(int frame[], int gen[], int rem[], int frame_len,
    int gen_len) {
    memcpy(rem, frame, frame_len * sizeof(int));
    for (int i = 0; i <= frame_len - gen_len; i++) {
        if (rem[i] == 1) {
            for (int j = 0; j < gen_len; j++) {
                rem[i + j] ^= gen[j];
            }
        }
    }
}

int main() {
    int frame[50] = {0}, gen[10] = {0}, rem[50] = {0}, frame_len,
```

```
Enter the degree of the data polynomial: 4
Enter the data polynomial coefficients (binary): 1
0
1
1
1
Enter the degree of the generator polynomial: 2
Enter the generator polynomial coefficients (binary): 1
0
0

Frame after appending zeros: 1011100
Remainder: 00
Transmitted Frame: 1011100

Enter the received frame: 1 0 1 1 1 0 0

No error detected. Data sent successfully.
```

Exercise-6

Aim:Write a Program to implement Sliding window protocol for Goback N.

Program:

```
// Online C compiler to run C program online
#include <stdio.h>
#include <string.h>

int main() {
    char sender[50], receiver[50];
    int i, winsize;

    // Input window size
    printf("Enter the window size: ");
    scanf("%d", &winsize);

    // Input the data to be sent
    printf("Enter the data to be sent: ");
    getchar(); // To consume the newline character left by scanf
```

```
    fgets(sender, sizeof(sender), stdin);
    sender[strcspn(sender, "\n")] = '\0'; // Remove trailing newline

    // Simulate sender window expansion
    printf("\nSender window expanded to store message.\n");
    printf("Message sent by the sender: %s\n", sender);

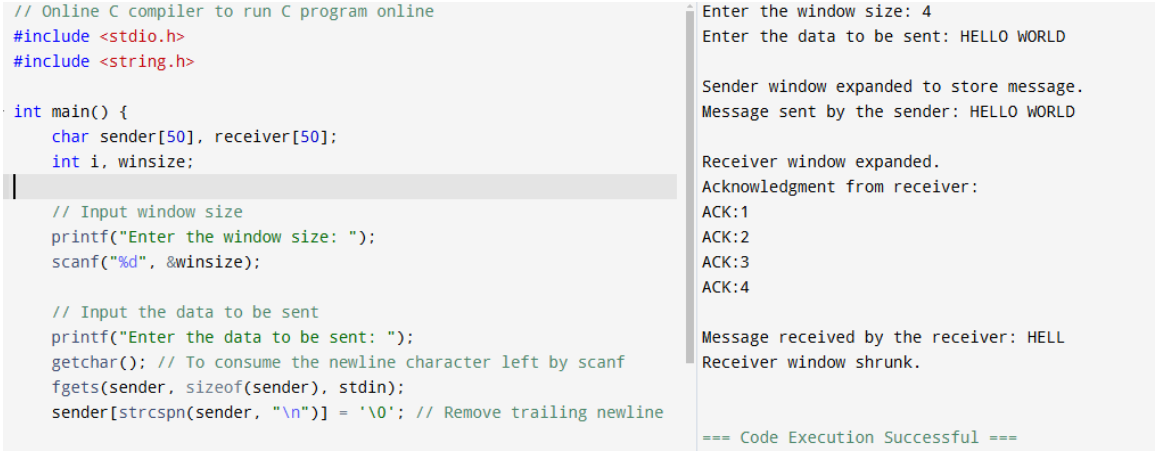
    // Transmitting data within the window size
    strncpy(receiver, sender, winsize);
    receiver[winsize] = '\0'; // Null terminate the receiver's string

    // Simulate receiver acknowledgment
    printf("\nReceiver window expanded.\n");
    printf("Acknowledgment from receiver:\n");
    for (i = 1; i <= winsize; i++) {
        printf("ACK:%d\n", i);
    }

    // Display the received message
    printf("\nMessage received by the receiver: %s\n", receiver);
    printf("Receiver window shrunk.\n");

    return 0;
}
```

Output:



```c
// Online C compiler to run C program online
#include <stdio.h>
#include <string.h>

int main() {
    char sender[50], receiver[50];
    int i, winsize;

    // Input window size
    printf("Enter the window size: ");
    scanf("%d", &winsize);

    // Input the data to be sent
    printf("Enter the data to be sent: ");
    getchar(); // To consume the newline character left by scanf
    fgets(sender, sizeof(sender), stdin);
    sender[strcspn(sender, "\n")] = '\0'; // Remove trailing newline
```

```
Enter the window size: 4
Enter the data to be sent: HELLO WORLD

Sender window expanded to store message.
Message sent by the sender: HELLO WORLD

Receiver window expanded.
Acknowledgment from receiver:
ACK:1
ACK:2
ACK:3
ACK:4

Message received by the receiver: HELL
Receiver window shrunk.


=== Code Execution Successful ===
```

Exercise-7

Aim:Write a Program to implement Sliding window protocol for Selective repeat.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define WINDOW_SIZE 4
#define TOTAL_FRAMES 10
void selective_repeat_protocol(int frames[], int total_frames, int window_size);
int main() {
    int frames[TOTAL_FRAMES];
    // Initialize frames with data
    for (int i = 0; i < TOTAL_FRAMES; i++) {
        frames[i] = i + 1; // Frame numbers from 1 to TOTAL_FRAMES
    }
    printf("Simulating Selective Repeat Protocol:\n");
    selective_repeat_protocol(frames, TOTAL_FRAMES, WINDOW_SIZE);
```

```
    return 0;
}
void selective_repeat_protocol(int frames[], int total_frames, int window_size) {
    int sent = 0; // Keeps track of sent frames
    int acked = 0; // Keeps track of acknowledged frames
    srand(time(0)); // Seed random number generator for simulating ACK/NAK
    while (acked < total_frames) {
        printf("\nSender's Window:\n");
        for (int i = sent; i < sent + window_size && i < total_frames; i++) {
            printf("Frame %d sent\n", frames[i]);
        }
        printf("\nReceiver's Response:\n");
        for (int i = sent; i < sent + window_size && i < total_frames; i++) {
            if (rand() % 5 == 0) { // Simulate NAK (20% chance of error)
                printf("Frame %d: NAK\n", frames[i]);
            } else {
                printf("Frame %d: ACK\n", frames[i]);
                acked++; // Increment acknowledged frames
            }
        }
        // Move the sender's window
        sent = acked;
    }
    printf("\nAll frames successfully transmitted and acknowledged.\n");
}
```

Output:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define WINDOW_SIZE 4
#define TOTAL_FRAMES 10
void selective_repeat_protocol(int frames[], int total_frames, int
    window_size);
int main() {
    int frames[TOTAL_FRAMES];

    // Initialize frames with data
    for (int i = 0; i < TOTAL_FRAMES; i++) {
        frames[i] = i + 1; // Frame numbers from 1 to TOTAL_FRAMES
    }
    printf("Simulating Selective Repeat Protocol:\n");
    selective_repeat_protocol(frames, TOTAL_FRAMES, WINDOW_SIZE);
    return 0;
}
void selective_repeat_protocol(int frames[], int total_frames, int
    window_size) {
    int sent = 0; // Keeps track of sent frames
    int acked = 0; // Keeps track of acknowledged frames
```

```
Simulating Selective Repeat Protocol:

Sender's Window:
Frame 1 sent
Frame 2 sent
Frame 3 sent
Frame 4 sent

Receiver's Response:
Frame 1: ACK
Frame 2: ACK
Frame 3: ACK
Frame 4: ACK

Sender's Window:
Frame 5 sent
Frame 6 sent
Frame 7 sent
Frame 8 sent

Receiver's Response:
Frame 5: ACK
```

```c
    }

    printf("\nReceiver's Response:\n");
    for (int i = sent; i < sent + window_size && i <
        total_frames; i++) {
        if (rand() % 5 == 0) { // Simulate NAK (20% chance of
            error)
            printf("Frame %d: NAK\n", frames[i]);
        } else {
            printf("Frame %d: ACK\n", frames[i]);
            acked++; // Increment acknowledged frames
        }
    }

    // Move the sender's window
    sent = acked;
}

printf("\nAll frames successfully transmitted and acknowledged
    .\n");
```

```
Frame 6 sent
Frame 7 sent
Frame 8 sent

Receiver's Response:
Frame 5: ACK
Frame 6: ACK
Frame 7: ACK
Frame 8: ACK

Sender's Window:
Frame 9 sent
Frame 10 sent

Receiver's Response:
Frame 9: ACK
Frame 10: ACK

All frames successfully transmitted and acknowledged.
```

Exercise-8

Aim:Write a Program to implement Stop and Wait Protocol

Program:

```c
#include <stdio.h>

#include <stdlib.h>

#include <time.h>


#define TIMEOUT 5
```

```
#define MAX_SEQ 1
#define TOT_PACKETS 8
#define inc(k) (k = (k < MAX_SEQ) ? k + 1 : 0)

typedef struct { int data; } packet;
typedef struct { int seq, err; packet info; } frame;
frame DATA;
int i = 1, DISCONNECT = 0; // Data and disconnect flag
char turn = 's'; // 's' for sender, 'r' for receiver
void from_network_layer(packet *p) { p->data = i++; }
void to_physical_layer(frame *f) { f->err = rand() % 4; DATA = *f; }
void from_physical_layer(frame *f) { *f = DATA; }
void to_network_layer(packet *p) { printf("RECEIVER: Packet %d received\n",
p->data); }
void sender();
void receiver();
int main() {
    srand(time(0));
    while (!DISCONNECT) {
        sender();
        receiver();
    }
    printf("\nDISCONNECTED\n");
    return 0;
}
```

```
void sender() {
    static int frame_to_send = 0, timer = 0;
    static frame s;
    if (turn == 's') {
        if (timer == 0) { // Send new frame
            packet p;
            from_network_layer(&p);
            s.info = p;
            s.seq = frame_to_send;
            printf("SENDER: Sending Packet %d, Seq %d\n", s.info.data, s.seq);
            to_physical_layer(&s);
            turn = 'r';
        } else if (timer++ >= TIMEOUT) { // Timeout
            printf("SENDER: Timeout, Resending Seq %d\n", frame_to_send);
            to_physical_layer(&s);
            turn = 'r';
            timer = 0;
        }
    }
}
void receiver() {
    if (turn == 'r') {
        frame r;
        from_physical_layer(&r);
        if (r.err == 0) { // Error in frame
```

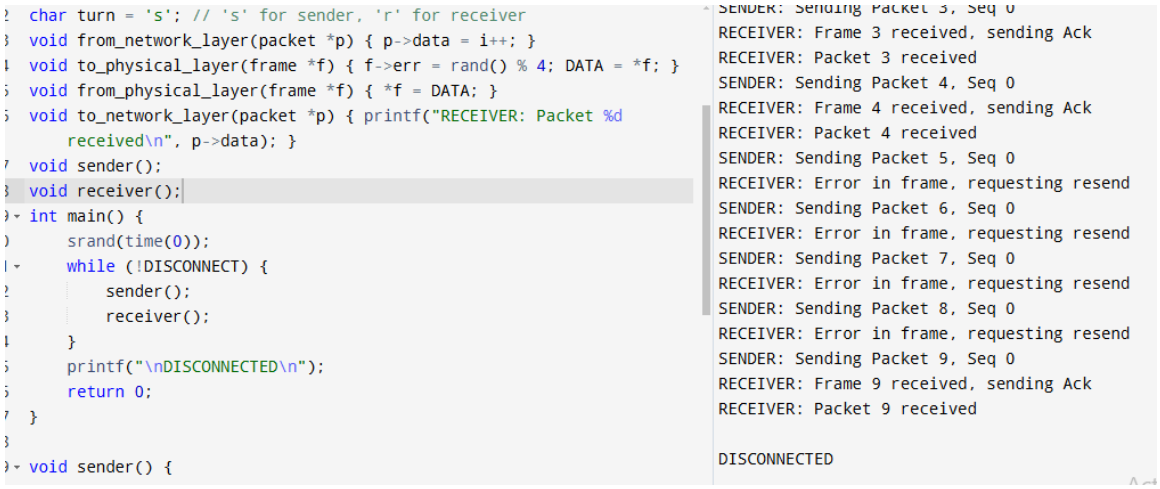
```
            printf("RECEIVER: Error in frame, requesting resend\n");
        } else { // Frame received correctly
            printf("RECEIVER: Frame %d received, sending Ack\n", r.info.data);
            to_network_layer(&r.info);
            if (i > TOT_PACKETS) DISCONNECT = 1; // Disconnect if all packets received
        }
        turn = 's';
    }
}
```

Output:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define TIMEOUT 5
#define MAX_SEQ 1
#define TOT_PACKETS 8
#define inc(k) (k = (k < MAX_SEQ) ? k + 1 : 0)
typedef struct { int data; } packet;
typedef struct { int seq, err; packet info; } frame;
frame DATA;
int i = 1, DISCONNECT = 0; // Data and disconnect flag
char turn = 's'; // 's' for sender, 'r' for receiver
void from_network_layer(packet *p) { p->data = i++; }
void to_physical_layer(frame *f) { f->err = rand() % 4; DATA = *f; }
void from_physical_layer(frame *f) { *f = DATA; }
void to_network_layer(packet *p) { printf("RECEIVER: Packet %d
    received\n", p->data); }
void sender();
void receiver();
int main() {
    srand(time(0));
    while (!DISCONNECT) {
```

```
SENDER: Sending Packet 1, Seq 0
RECEIVER: Frame 1 received, sending Ack
RECEIVER: Packet 1 received
SENDER: Sending Packet 2, Seq 0
RECEIVER: Frame 2 received, sending Ack
RECEIVER: Packet 2 received
SENDER: Sending Packet 3, Seq 0
RECEIVER: Frame 3 received, sending Ack
RECEIVER: Packet 3 received
SENDER: Sending Packet 4, Seq 0
RECEIVER: Frame 4 received, sending Ack
RECEIVER: Packet 4 received
SENDER: Sending Packet 5, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 6, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 7, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 8, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 9, Seq 0
RECEIVER: Frame 9 received, sending Ack
```

Activate Win
Go to Settings to



```
char turn = 's'; // 's' for sender, 'r' for receiver
void from_network_layer(packet *p) { p->data = i++; }
void to_physical_layer(frame *f) { f->err = rand() % 4; DATA = *f; }
void from_physical_layer(frame *f) { *f = DATA; }
void to_network_layer(packet *p) { printf("RECEIVER: Packet %d
    received\n", p->data); }
void sender();
void receiver();
int main() {
    srand(time(0));
    while (!DISCONNECT) {
        sender();
        receiver();
    }
    printf("\nDISCONNECTED\n");
    return 0;
}
void sender() {
```

```
SENDER: Sending Packet 3, Seq 0
RECEIVER: Frame 3 received, sending Ack
RECEIVER: Packet 3 received
SENDER: Sending Packet 4, Seq 0
RECEIVER: Frame 4 received, sending Ack
RECEIVER: Packet 4 received
SENDER: Sending Packet 5, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 6, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 7, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 8, Seq 0
RECEIVER: Error in frame, requesting resend
SENDER: Sending Packet 9, Seq 0
RECEIVER: Frame 9 received, sending Ack
RECEIVER: Packet 9 received

DISCONNECTED
```

Exercise-9

Aim:Write a program for congestion control using leaky bucket algorithm

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // For sleep()
#define NOF_PACKETS 10
int generate_random(int a) {
    int rn = (rand() % 10) % a;
    return rn == 0 ? 1 : rn;
}
int main() {
    int packet_sz[NOF_PACKETS], i, clk, b_size, o_rate, p_sz_rm = 0, op;
    // Generate random packet sizes
    for (i = 0; i < NOF_PACKETS; ++i)
        packet_sz[i] = generate_random(6) * 10;
    printf("Generated Packet Sizes:\n");
```

```
    for (i = 0; i < NOF_PACKETS; ++i)
        printf("packet[%d]: %d bytes\n", i, packet_sz[i]);
    printf("\nEnter the Output rate: ");
    scanf("%d", &o_rate);
    printf("Enter the Bucket Size: ");
    scanf("%d", &b_size);
    // Process each packet
    for (i = 0; i < NOF_PACKETS; ++i) {
        if ((packet_sz[i] + p_sz_rm) > b_size) {
            if (packet_sz[i] > b_size) {
                printf("\nIncoming packet size (%d bytes) exceeds bucket capacity
(%d bytes) - PACKET REJECTED\n",
                        packet_sz[i], b_size);
            } else {
                printf("\nBucket capacity exceeded - PACKETS REJECTED!!\n");
            }
        } else {
            p_sz_rm += packet_sz[i];
            printf("\nIncoming Packet size: %d bytes", packet_sz[i]);
            printf("\nBytes remaining to Transmit: %d", p_sz_rm);
            int p_time = generate_random(4) * 10;
            printf("\nEstimated Transmission Time: %d units\n", p_time);
            for (clk = 10; clk <= p_time; clk += 10) {
                sleep(1);
                if (p_sz_rm > 0) {
```

```
                    if (p_sz_rm <= o_rate) {
                        op = p_sz_rm;
                        p_sz_rm = 0;
                    } else {
                        op = o_rate;
                        p_sz_rm -= o_rate;
                    }
                    printf("Packet of size %d transmitted --- Bytes Remaining: %d\n", op, p_sz_rm);
                } else {
                    printf("Time left for transmission: %d units\n", p_time - clk);
                    printf("No packets to transmit!!\n");
                    break;
                }
            }
        }
    }
    return 0;
}
```

Output:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h> // For sleep()

#define NOF_PACKETS 10

int generate_random(int a) {
    int rn = (rand() % 10) % a;
    return rn == 0 ? 1 : rn;
}

int main() {
    int packet_sz[NOF_PACKETS], i, clk, b_size, o_rate, p_sz_rm = 0,
        op;

    // Generate random packet sizes
    for (i = 0; i < NOF_PACKETS; ++i)
        packet_sz[i] = generate_random(6) * 10;

    printf("Generated Packet Sizes:\n");
    for (i = 0; i < NOF_PACKETS; ++i)
        printf("packet[%d]: %d bytes\n", i, packet_sz[i]);
```

```
Generated Packet Sizes:
packet[0]: 30 bytes
packet[1]: 10 bytes
packet[2]: 10 bytes
packet[3]: 50 bytes
packet[4]: 30 bytes
packet[5]: 50 bytes
packet[6]: 10 bytes
packet[7]: 20 bytes
packet[8]: 30 bytes
packet[9]: 10 bytes

Enter the Output rate: 4
Enter the Bucket Size: 2

Incoming packet size (30 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED
Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
```

Activate Windows
Go to Settings to activate Windows.

```c
}

int main() {
    int packet_sz[NOF_PACKETS], i, clk, b_size, o_rate, p_sz_rm = 0,
        op;

    // Generate random packet sizes
    for (i = 0; i < NOF_PACKETS; ++i)
        packet_sz[i] = generate_random(6) * 10;

    printf("Generated Packet Sizes:\n");
    for (i = 0; i < NOF_PACKETS; ++i)
        printf("packet[%d]: %d bytes\n", i, packet_sz[i]);

    printf("\nEnter the Output rate: ");
    scanf("%d", &o_rate);
    printf("Enter the Bucket Size: ");
    scanf("%d", &b_size);

    // Process each packet
    for (i = 0; i < NOF_PACKETS; ++i) {
```

```
Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (50 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (30 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (50 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (20 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED
```

Activate Windows
Go to Settings to activate Windows.

```c
    // Process each packet
    for (i = 0; i < NOF_PACKETS; ++i) {
        if ((packet_sz[i] + p_sz_rm) > b_size) {
            if (packet_sz[i] > b_size) {
                printf("\nIncoming packet size (%d bytes) exceeds
                    bucket capacity (%d bytes) - PACKET REJECTED\n",
                        packet_sz[i], b_size);
            } else {
                printf("\nBucket capacity exceeded - PACKETS
                    REJECTED!!\n");
            }
        } else {
            p_sz_rm += packet_sz[i];
            printf("\nIncoming Packet size: %d bytes", packet_sz[i]
                );
            printf("\nBytes remaining to Transmit: %d", p_sz_rm);

            int p_time = generate_random(4) * 10;
            printf("\nEstimated Transmission Time: %d units\n",
                p_time);
```

```
REJECTED

Incoming packet size (30 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (50 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (20 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (30 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED

Incoming packet size (10 bytes) exceeds bucket capacity (2 bytes) - PACKET
    REJECTED
```

Activate Windows
Go to Settings to activate Windows.

Exercise-10

Aim:Write a Program to implement Dijkstra‘s algorithm to compute the Shortest path through a graph.

Program:

#include <stdio.h>

```
#define INF 999 // Infinity value for unconnected nodes


int main() {
    int cost[10][10], dist[10], visited[10] = {0}, pre[10];
    int n, source, target, start, min, m, d, i, j, path[10];

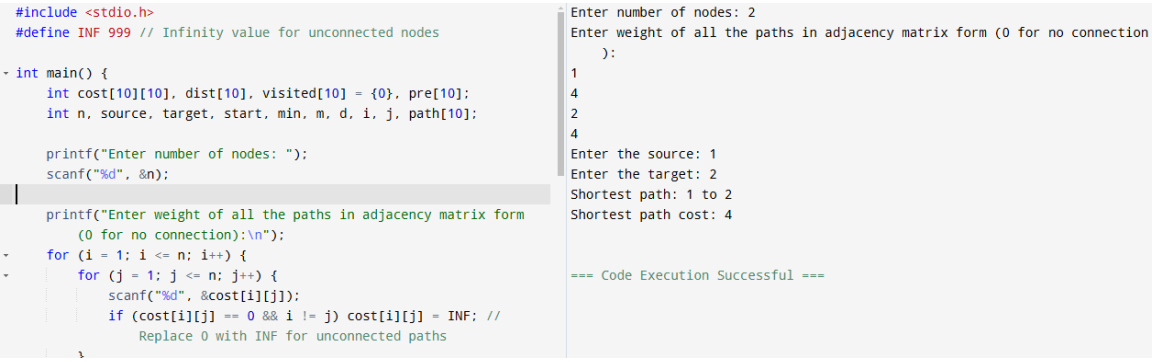

    printf("Enter number of nodes: ");
    scanf("%d", &n);


    printf("Enter weight of all the paths in adjacency matrix form (0 for no connection):\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j) cost[i][j] = INF; // Replace 0 with INF for unconnected paths
        }
    }
    printf("Enter the source: ");
    scanf("%d", &source);
    printf("Enter the target: ");
    scanf("%d", &target);
    // Initialize distances and predecessors
    for (i = 1; i <= n; i++) {
        dist[i] = INF;
```

```
        pre[i] = -1;
    }
    dist[source] = 0; // Distance to source is 0
    // Dijkstra's Algorithm
    start = source;
    while (visited[target] == 0) {
        min = INF;
        m = 0;
        for (i = 1; i <= n; i++) {
            d = dist[start] + cost[start][i];
            if (d < dist[i] && visited[i] == 0) {
                dist[i] = d;
                pre[i] = start;
            }
            if (dist[i] < min && visited[i] == 0) {
                min = dist[i];
                m = i;
            }
        }
        start = m;
        visited[m] = 1;
    }
    // Reconstructing the shortest path
    j = 0;
    start = target;
```

```
    while (start != -1) {
        path[j++] = start;
        start = pre[start];
    }
    // Output the path
    printf("Shortest path: ");
    for (i = j - 1; i >= 0; i--) {
        printf("%d", path[i]);
        if (i > 0) printf(" to ");
    }
    printf("\nShortest path cost: %d\n", dist[target]);
    return 0;
}
```

Output:



```
#include <stdio.h>
#define INF 999 // Infinity value for unconnected nodes

int main() {
    int cost[10][10], dist[10], visited[10] = {0}, pre[10];
    int n, source, target, start, min, m, d, i, j, path[10];

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter weight of all the paths in adjacency matrix form
        (0 for no connection):\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0 && i != j) cost[i][j] = INF; //
                Replace 0 with INF for unconnected paths
```

```
Enter number of nodes: 2
Enter weight of all the paths in adjacency matrix form (0 for no connection
    ):
1
4
2
4
Enter the source: 1
Enter the target: 2
Shortest path: 1 to 2
Shortest path cost: 4


=== Code Execution Successful ===
```

Exercise-11

Aim:Write a Program to implement Distance vector routing algorithm by obtaining routing table at each node

(Take an example subnet graph with weights indicating delay between nodes).

Program:

```
#include<stdio.h>
```

```
struct node {
    unsigned dist[20]; // Distance array
    unsigned from[20]; // Source node array
} rt[10];
int main() {
    int costmat[20][20], nodes, i, j, k, count = 0;
    printf("Enter the number of nodes: ");
    scanf("%d", &nodes); // Number of nodes
    printf("Enter the cost matrix:\n");
    for (i = 0; i < nodes; i++) {
        for (j = 0; j < nodes; j++) {
            scanf("%d", &costmat[i][j]);
            costmat[i][i] = 0; // Distance to itself is always zero
            rt[i].dist[j] = costmat[i][j]; // Initialize distance to cost matrix
            rt[i].from[j] = j; // Initialize the source
        }
    }
    // Distance Vector Algorithm
    do {
        count = 0;
        for (i = 0; i < nodes; i++) { // For each router
            for (j = 0; j < nodes; j++) { // For each destination
                for (k = 0; k < nodes; k++) { // Check through every node as an
intermediate
                    if (rt[i].dist[j] > costmat[i][k] + rt[k].dist[j]) {
```

```
                        // Update distance if a shorter path is found
                        rt[i].dist[j] = costmat[i][k] + rt[k].dist[j];
                        rt[i].from[j] = k;
                        count++;
                    }
                }
            }
        }
    } while (count != 0); // Repeat until no updates
    // Print the routing table for each router
    for (i = 0; i < nodes; i++) {
        printf("\n\nRouting Table for Router %d:\n", i + 1);
        printf("Destination\tNext Hop\tDistance\n");
        for (j = 0; j < nodes; j++) {
            printf("%d\t\t%d\t\t%d\n", j + 1, rt[i].from[j] + 1, rt[i].dist[j]);
        }
    }
    printf("\n");
    return 0;
}
```

Output:

```c
#include<stdio.h>
struct node {
    unsigned dist[20]; // Distance array
    unsigned from[20]; // Source node array
} rt[10];

int main() {
    int costmat[20][20], nodes, i, j, k, count = 0;

    printf("Enter the number of nodes: ");
    scanf("%d", &nodes); // Number of nodes

    printf("Enter the cost matrix:\n");
    for (i = 0; i < nodes; i++) {
        for (j = 0; j < nodes; j++) {
            scanf("%d", &costmat[i][j]);
            costmat[i][i] = 0; // Distance to itself is always zero
            rt[i].dist[j] = costmat[i][j]; // Initialize distance to
                cost matrix
            rt[i].from[j] = j; // Initialize the source
        }
    }
```

```
Enter the number of nodes: 3
Enter the cost matrix:
2
4
3
6
4
7
4
8
2

Routing Table for Router 1:
Destination Next Hop    Distance
1       1       0
2       2       4
3       3       3

Routing Table for Router 2:
Destination Next Hop    Distance
```

Activate Windows
Go to Settings to activa

```c
    // Distance Vector Algorithm
    do {
        count = 0;
        for (i = 0; i < nodes; i++) { // For each router
            for (j = 0; j < nodes; j++) { // For each destination
                for (k = 0; k < nodes; k++) { // Check through every
                    node as an intermediate
                    if (rt[i].dist[j] > costmat[i][k] + rt[k]
                        .dist[j]) {
                        // Update distance if a shorter path is
                            found
                        rt[i].dist[j] = costmat[i][k] + rt[k]
                            .dist[j];
                        rt[i].from[j] = k;
                        count++;
                    }
                }
            }
        }
    } while (count != 0); // Repeat until no updates
```

```
Destination Next Hop    Distance
1       1       0
2       2       4
3       3       3


Routing Table for Router 2:
Destination Next Hop    Distance
1       1       6
2       2       0
3       3       7


Routing Table for Router 3:
Destination Next Hop    Distance
1       1       4
2       2       8
3       3       0



=== Code Execution Successful ===
```